32 INJECTING & USING A MONGOOSE MODEL IN A SERVICE — @InjectModel, Model<T>, async CRUD

theory/31 covered how @Schema()/@Prop() turn a class into a Mongoose Schema object. This file covers the next step: how student.service.ts actually gets hold of that schema at runtime and uses it to read/write data — the part where a decorated class becomes a working database client inside a Nest provider.

STEP 0 — REGISTRATION (the piece that makes injection possible)

```
// student.module.ts
MongooseModule.forFeature([{ name: Student.name, schema: StudentSchema }])
```

forFeature() takes the StudentSchema built in theory/31 and registers it with Nest's DI container under the token 'StudentModel' internally (derived from Student.name, i.e. the string "Student"). Nothing below works without this line — it's what makes a Model injectable at all, the same way a class needs @Injectable() before it can be constructor- injected anywhere (theory/07-dependency-injection.txt).

@InjectModel(Student.name) — PARAMETER DECORATOR

```
constructor(@InjectModel(Student.name) private studentModel: Model<StudentDocument>) {}
```

Same category of decorator as @Body()/@Param() in a controller (theory/06-decorators.txt) — it doesn't transform anything itself, it just tells Nest's DI container which provider token to resolve for this constructor parameter. @InjectModel(Student.name) says "give me whatever forFeature() registered under the name 'Student'."

Student.name is used instead of the literal string 'Student' so that renaming the class updates the injection token automatically — see theory/06 for the same reasoning applied to Student.name in forFeature() itself.

Model<StudentDocument> is the TYPE of what gets injected: a generic Mongoose Model, parameterized with StudentDocument (Student's own fields + Mongoose's Document methods, from theory/31). This is what gives you typed autocomplete on studentModel.find(), .create(), etc., checked against Student's actual fields.

WHY EVERY METHOD RETURNS A Promise

```
async findAll(): Promise<StudentDocument[]> {
  return this.studentModel.find().exec();
}
```

task.service.ts's methods are plain, synchronous — they read/write a `private tasks: Task[]` array sitting in memory, so a value is always available instantly. student.service.ts has no such array; every read or write is a real network round-trip to a remote MongoDB cluster (theory/30-mongodb-connection.txt). JavaScript can't return a value that doesn't exist yet, so Mongoose gives you back a Promise instead — a placeholder for "the result, once the round-trip finishes."

async/await is what lets calling code treat that Promise as if it were a synchronous value:

```
async findOne(id: string): Promise<StudentDocument> {
  const student = await this.studentModel.findById(id).exec();
  ...
}
```

Without `await`, `student` would be a Promise object, not the document itself — you'd be checking `if (!somePromiseObject)`, which is always truthy, never the actual null/found result.

.exec() itself is a Mongoose convention: query builder methods like .find()/.findById() return a "thenable" query object immediately (lets you chain .sort()/.limit()/etc. before it runs), and .exec() is what actually fires the query and returns a real Promise.

THE THREE METHODS BUILT SO FAR

```
async create(dto: CreateStudentDto): Promise<StudentDocument> {
  const newStudent = new this.studentModel(dto);
  return newStudent.save();
}
```

`new this.studentModel(dto)` builds a document IN MEMORY — Student's @Prop({ required: true }) rules (theory/31) are checked at this point, before anything touches the network. `.save()` is what actually sends an insert to MongoDB and returns the saved doc, now including the _id and timestamps Mongo/Mongoose generated.

```
async findAll(): Promise<StudentDocument[]> {
  return this.studentModel.find().exec();
}
```

`.find()` with no filter argument means "match everything" — same idea as an empty WHERE clause. Returns an array, hence Promise<StudentDocument[]> rather than a single document.

```
async findOne(id: string): Promise<StudentDocument> {
  const student = await this.studentModel.findById(id).exec();
  if (!student) {
    throw new NotFoundException(`Student ${id} not found`);
  }
  return student;
}
```

`.findById(id)` looks up by Mongo's _id field specifically — not an array index like task.service.ts's findOne(id) does. Mongoose resolves `null` (not an error) when nothing matches, so the `if (!student)` check is what turns "found nothing" into an actual thrown NotFoundException — otherwise the controller would happily return `null` with a 200 status, which is a worse contract for API consumers than a proper 404 (theory/24-exception-handling.txt covers how a thrown NotFoundException becomes an HTTP response).

DTO IN, SCHEMA-BACKED MODEL OUT

create()'s parameter is a CreateStudentDto (theory/13-dto.txt, theory/15-class-validator-and-class-transformer.txt) — validated by the global ValidationPipe before this method ever runs. The method body then hands that already-validated dto straight to `new this.studentModel(dto)`, where Student's OWN @Prop() rules

(required/unique) apply again — a second, independent validation layer at the database level. Two different concerns, two different decorator families, covered by theory/31's "HOW THIS DIFFERS FROM DTO DECORATORS" section.

★ RULE OF THUMB

@InjectModel(X.name) is DI, not magic — it only works because MongooseModule.forFeature() registered that exact model name first. Model<XDocument> is a typed handle to a real MongoDB collection. Every method touching it must be async and every call to it awaited, because unlike an in-memory array, the data isn't already sitting in the process — it's a network call away. `.exec()` is what turns a Mongoose query builder into a real Promise. A `null` result from Mongoose isn't an error by itself — checking for it and throwing NotFoundException is what makes "not found" behave like a proper HTTP 404 instead of a silent empty success.

See also: theory/31-mongoose-schema-decorators.txt, theory/30-mongodb-connection.txt, theory/07-dependency-injection.txt, theory/13-dto.txt, theory/24-exception-handling.txt